

PROJECT REPORT

Autonomous Robots



Ostbayerische Technische Hochschule
Amberg-Weiden

MAPPING, PERCEPTION, PATH PLANNING AND NAVIGATION

Submitted by

Sathwik Nagasundra Sharma

Siddhant Tiwari

Zicheng Cai

Nishith Kumar Alugandula

Course

Autonomous Robots

Ostbayerische Technische Hochschule Amberg-Weiden

Department of Electrical Engineering, Media and Computer Science

Abstract

This report documents the design and implementation of an autonomous navigation controller for a RosBot robot in the Webots simulation Worlds. The robot must first reach the blue pillar and then the yellow pillar while avoiding green hazardous ground, normal obstacles, too-narrow passages and floating wall structures. The solution uses only onboard IMU sensors. The controller is organised into hardware and odometry, perception, occupancy-grid mapping, global A* planning, local Dynamic Window Approach motion control, reactive safety, replanning and mission-level state control.

1 Project Objective

The goal of the project is to implement autonomous navigation for a RosBot in Webots Mazes. The robot starts in an unknown maze and must complete the mission in the following order:

1. build a useful map using onboard sensors,
2. detect and localise the blue and yellow pillars,
3. navigate from the start area to the blue pillar,
4. after reaching blue, navigate to the yellow pillar,
5. avoid the green ground region and all physical obstacles.

The implementation is designed for dynamic mazes, so the robot should work even when it is placed at a different initial pose. The robot does not assume a predefined map and does not use Webots Supervisor position data.

2 Project Structure

The codebase is divided into clear layers. Each file has one main responsibility, which makes the controller easier to debug and extend.

File	Main role
<code>main_mazeX.py</code>	Creates robot, perception, navigator and mission objects.
<code>my_robot.py</code>	Device setup wrapper, odometry, coordinate conversion and low-level motion.
<code>setup.py</code>	Enables motors, sensors, cameras, lidar and IMU devices.
<code>perception.py</code> <code>map.py</code>	Interprets RGB, depth and lidar data. Maintains the occupancy grid, frontiers and path-planning map.
<code>astar_2_spline.py</code>	Runs A* search and smooths the final path.
<code>navigation.py</code>	Executes plan-follow-replan navigation with safety checks.
<code>mission.py</code>	Controls initial scan, exploration and final blue-to-yellow traversal.
<code>CONSTANTS.py</code>	Stores tunable parameters and map marker values.
<code>utils.py</code>	Provides helper functions such as colour segmentation and Bresenham lines.

Table 1: Main files used in the autonomous robot controller.

3 System Architecture

The controller follows a layered architecture. The hardware layer reads sensors and controls the motors. The perception layer converts raw camera and lidar data into useful information such as coloured pillars, green hazards and depth obstacles. The map layer stores this information in an occupancy grid. The navigation layer plans and follows paths, while the mission layer decides which phase the robot is currently executing.

Layer	Responsibility
Hardware / pose	Sensor access, wheel motors, encoder odometry and compass heading.
Perception	HSV colour detection, pillar localisation, green-floor detection and depth-obstacle projection.
Mapping	Log-odds occupancy grid, special cell marking, frontier detection and obstacle inflation.
Planning	Global A* path search with wall-clearance penalty and B-spline smoothing.
Navigation	Waypoint following, Dynamic Window Approach, emergency checks and replanning.
Mission	Initial scan, exploration, pillar discovery and final traversal.

Table 2: Layered system architecture.

4 Occupancy Grid Mapping

The map is stored as a grid of cell values. Normal obstacles, unknown cells, free space, pillars, green hazards and floating walls have different values so that they can be visualised and handled differently during planning.

For each lidar endpoint, a Bresenham line is drawn from the robot cell to the hit cell. Cells before the endpoint are updated as free space, while the final cell is updated as occupied. Log-odds values are used so repeated observations increase the confidence of the map.

```

for map_target in lidar_map_points
:
    points = bresenham_line(
        robot_pos, map_target)
    for x, y in points[:-1]:
        log_odds[y, x] -= 0.36
        # free cells
    x, y = points[-1]
    log_odds[y, x] += 0.85
    # obstacle cell

```

Special cells such as green carpet and floating

walls are protected. This means that later lidar readings cannot accidentally erase hazards or depth-detected obstacles.

5 Perception

The perception module converts sensor readings into semantic information. It uses HSV colour segmentation because colour thresholds are easier to tune in HSV than in RGB.

Pillar detection

Blue and yellow pillars are detected by thresholding their HSV colour masks. If the coloured blob has enough pixels, the object is accepted as a pillar. The horizontal centroid gives the bearing and the median depth over the blob gives the range.

$$x_r = r \cos(\beta), \quad (1)$$

$$y_r = r \sin(\beta) \quad (2)$$

Here r is the median depth and β is the bearing from the image centre. The local point is transformed into the world frame using the robot pose and then stored as a map coordinate.

Green hazard detection

The green area is a forbidden region. The robot checks the lower central part of the RGB image for green pixels and uses depth to verify that the green region is close. Detected green pixels are projected to the ground plane and stamped into the map as **GREEN_CARPET**.

During navigation these cells are treated as hard obstacles.

Depth obstacle detection

A horizontal lidar can miss objects that are too low, flat on the floor or floating above the scan plane. The depth camera helps detect such obstacles by projecting depth points into the ground plane. Ground-level obstacles are marked as normal obstacles, while elevated obstacles are marked as floating walls.

6 Exploration Strategy

At the start, the robot has no complete map. The mission first performs a slow in-place scan. The 360-degree turn is divided into small slices, and the robot waits for the sensors to settle before adding lidar data to the grid. This produces a cleaner initial map.

After that, the robot uses frontier exploration. A frontier is a free cell next to an unknown cell. Frontier clusters represent useful places to explore because they lie at the boundary of known and unknown space.

```
regions = map.detect_frontiers()
for region in regions:
    centroid = mean(region)
    if planner(robot_position,
               centroid):
        navigate_to(centroid)
```

The robot continues selecting reachable frontier centroids until both pillars are localised.

7 A* Path Planning

Global path planning is performed using A* search on the occupancy grid. The planner uses eight-connected motion with two-pixel steps. Straight moves are cheaper than diagonal moves, and the heuristic is based on Euclidean distance.

$$f(n) = g(n) + h(n) \quad (3)$$

The planner also adds a clearance penalty near walls. This makes the robot prefer routes that are slightly longer but safer, instead of driving directly beside obstacles.

$$C_{penalty}(n) = \begin{cases} k \left(1 - \frac{d(n)}{d_{safe}}\right), & d(n) < d_{safe} \\ 0, & d(n) \geq d_{safe} \end{cases} \quad (4)$$

After the path is found, B-spline smoothing is applied. The smoothed path is easier for the local controller to follow because it reduces sharp grid-based turns.

8 Navigation and Replanning

The navigation module is the executor. It receives a goal cell from the mission, asks the planner for a path, converts the dense path into waypoints, faces the first waypoint, and then follows the path.

```
for attempt in range(
    NAV_MAX_REPLANS):
    start = robot.get_map_position()
    path = planner(start, goal)
```

```

if not path:
    nudge_rotate()
    continue
waypoints = to_waypoints(path)
face_path(waypoints)
result = follow_waypoints(
    waypoints)
if result == "reached":
    return "reached"
# otherwise update map and
replan

```

This plan–follow–replan strategy is important because the map changes while the robot moves. If a new obstacle, green patch or floating wall is detected, the current path becomes invalid and a new path must be planned.

9 Dynamic Window Approach

The local motion controller uses a Dynamic Window Approach style method. Several linear and angular velocity candidates are sampled. Each candidate predicts a short future motion and receives a score based on heading alignment, distance to the target and speed. The best candidate is converted into left and right wheel velocities.

$$v_l = \frac{v - \omega L/2}{R}, \quad (5)$$

$$v_r = \frac{v + \omega L/2}{R} \quad (6)$$

Here v is the linear velocity, ω is angular velocity, L is axle length and R is wheel radius.

10 Reactive Safety

Reactive safety is checked during every navigation tick. The navigator stops and requests replanning when a dangerous condition is detected.

- green ground is close in front of the robot,
- lidar detects an obstacle inside the front cone,
- range sensors detect a near obstacle,
- depth camera detects an obstacle at collision height,
- the robot is stuck and no longer making progress.

When the robot is stuck, the map can stamp the blocked region in front of the robot. The next A* search then avoids that same location. Recovery mainly uses in-place rotation instead of reversing, so the robot changes viewpoint and tries a new path safely.

11 Simulation Results

The mission timing was measured in two stages: from the start of the simulation until the blue pillar was reached, and from the blue pillar until the yellow pillar was reached. A demonstration video playlist is available at this link.

Maze	Start–Blue	Blue–Yellow	Total
Maze 1	03:26	01:11	04:37
Maze 2	01:49	02:02	03:52
Maze 3	01:40	01:46	03:26
Maze 4	01:40	00:50	02:30
Maze 5	02:25	00:50	03:15

Table 3: Simulation timing results for the autonomous robot mission.

12 Maze-wise Special Approach

The same basic pipeline is used in all mazes: initial scanning, frontier exploration, pillar localisation, A* planning, DWA-based local motion and reactive replanning. The maze-wise changes are not separate hardcoded maps; they are small parameter and recovery changes added to handle the special behaviour of each maze. A separate Maze 2 controller archive was not provided, so Maze 2 is reported as the common baseline version.

Maze 1: conservative reference behaviour

Maze 1 uses a more conservative reference setup. The map version includes a confidence-grid idea, where repeated hits increase obstacle confidence and free observations reduce it. Important parameters are `CONFIDENCE_INITIAL=100`, `CONFIDENCE_HIT_INC=30`, `CONFIDENCE_MISS_DEC=3`, and `CONFIDENCE_WALL_THRESHOLD=110`. The navigation also uses denser waypoint tracking with `NAV_WAYPOINT_STRIDE=2` and more replanning attempts with `NAV_MAX_REPLANS=50`.

The reason for this setup is to make the robot less sensitive to one wrong lidar reading. Dense waypoints give better path tracking in short turns, and extra replans help if exploration is temporarily blocked by noisy map cells. The pillar update is also stricter with `COLUMN_REFINE_MIN_PIXELS=1000`, so a weak camera detection does not replace a better stored pillar position.

Maze 2: common baseline controller

No separate Maze 2 code archive was attached, so the report treats Maze 2 as the common baseline controller. The baseline uses log-odds occupancy-grid mapping, frontier exploration, clearance-aware A*, B-spline path smoothing, DWA local motion and normal safety checks. The typical settings are `NAV_WAYPOINT_STRIDE=6`, `NAV_GOAL_REACH_PX=6`, `NAV_MAX_REPLANS=20`, and `NAV_MAP_UPDATE_MIN_DIST_M=0.05`.

This balanced setup is useful when no special behaviour is required. It gives stable map updates, avoids over-correcting the path, and keeps the controller general enough to work without maze-specific assumptions.

Maze 3: accurate pillar reaching and stronger recovery

Maze 3 is tuned for precise final positioning near the coloured pillars. The main special value is `COLUMN_REACH_PX=2`, which means the robot only counts a pillar as reached when its map position is very close to the stored pillar cell. The recovery behaviour is also stronger, using `EMERGENCY_BACKUP_MS=1000`. When the robot gets stuck, the map stamps a small blocked area in front of the robot using approximately `forward_m=0.15` and `radius_m=0.05`.

The reason for this tuning is that early success detection can be wrong when the robot is near a pillar but not actually close enough. The longer emergency backup gives more space to recover from narrow or too-narrow passages. The blocked-cell stamping forces the next A*

search to avoid the exact place where the robot became stuck.

Maze 4: close-pillar filtering and careful local marking

Maze 4 adds a close-pillar dead-zone filter using `DEPTH_DEAD_ZONE_PIXELS=30`. If the robot is already very close to a known pillar, the code avoids updating the stored pillar position again. Maze 4 also uses a larger pillar reach tolerance, `COLUMN_REACH_PX=10`, a shorter emergency backup, `EMERGENCY_BACKUP_MS=500`, and a smaller blocked-area stamp of about `forward_m=0.10` and `radius_m=0.01`. The closed-loop turn speed is reduced to `speed=2.0`.

This is useful because close-up depth readings can be unstable and may corrupt a good pillar estimate. The smaller blocked-cell stamp is safer in tight maze sections because it does not accidentally close a valid narrow passage. The slower turn speed reduces overshoot during recovery and alignment.

Maze 5: faster traversal with looser tolerances

Maze 5 is tuned for faster movement and longer routes. It uses higher motor values, `MOTOR_VELOCITY_FORWARD=8`, `MOTOR_VELOCITY_TURN=8`, and `MOTOR_VELOCITY_BACKWARD=-8`. The waypoint and goal tolerances are looser: `NAV_WAYPOINT_STRIDE=8`, `NAV_GOAL_REACH_PX=12`, and `COLUMN_REACH_PX=7`. The step budget is increased to `NAV_MAX_STEPS_PER_GOAL=600000`. Maze 5 also reduces `PATH_MIN_LENGTH_M=0.55` and uses stricter depth filtering with `DEPTH_OBST_MIN_`

`M=0.1` and `DEPTH_OBST_MIN_POINTS=10`.

The reason is to finish a longer maze faster without wasting time on very small path corrections. The larger step budget prevents long paths from failing early. The stricter depth filter reduces false obstacle detections from very near or noisy depth pixels, while the looser reach tolerance prevents oscillation around the final pillar.

Overall, the robot still follows the same autonomous method in every maze. The maze-wise changes only tune safety, mapping confidence, recovery distance, turning speed and arrival tolerance according to the difficulty of each maze.

13 Conclusion

This project implements a complete autonomous navigation pipeline for a RosBot robot in Webots. The system builds an occupancy grid, detects blue and yellow pillars, marks green hazardous ground, explores unknown space using frontiers, plans global paths using clearance-aware A*, follows paths using local motion control and replans whenever the environment becomes unsafe. The modular design keeps perception, mapping, planning, navigation and mission logic separate, which makes the solution easier to understand, test and improve.